USTO-MB University of Science and Technology of Oran – Mohamed Boudiaf

Parallel Fractal Zoom Movie in Python: Threads, Virtual Machines, Docker Containers and a Physical LAN

Course: Intensive Computing

by: Bellatreche Mohamed Amine, Rezaiguia Soltane

Supervisor: Prof. Dairi Abdelkader

Academic year: 2026–2027 5th year ING, Data Science

1 Introduction

The computation of the Mandelbrot set is an archetype of embarrassingly parallel problems with severe computational load imbalance. Defined on the complex plane, each pixel represents a constant $c = c_r + ic_i$. Starting from $z_0 = 0$, the sequence $z_{n+1} = z_n^2 + c$ is iterated until either $|z_n| > 2$ (escape) or an iteration threshold M is reached. While individual pixels require zero inter-pixel communication during calculation, the work per pixel is inherently irregular: exterior pixels escape in a handful of iterations, whereas interior points and fine boundary filaments iterate to the limit M.

In an animated fractal zoom, a continuous sequence of frames is rendered as the view window contracts toward an interesting boundary point. To preserve visual detail across magnifying scales, the iteration limit must increase dynamically with the zoom depth. Consequently, parallel work distribution cannot assume uniform cost either across image regions or across consecutive frames.

The objective of this project is to implement, evaluate, and compare parallel execution models in Python. We develop a serial baseline (V0), a shared-memory multi-threaded engine using Numba with OpenMP (V1), and three message-passing distributed strategies using `mpi4py` (S1, S2, and S3). Under the scope of this project phase, all implementations are benchmarked and verified locally on an AMD Ryzen 5 multicore processor. Multi-node cluster deployments across bridged Virtual Machines (V2), Docker containers (V3), and a physical lab LAN (V4) are marked as deferred to a subsequent phase.

2 Implementation

2.1 Shared Computational Kernel and Geometry

All parallel and serial versions share a single computational kernel implemented in `src/fractal_core.py`. Strict determinism is enforced across all versions by disabling `fastmath`.

To prevent floating-point discrepancies across subgrid boundaries, pixel coordinates are computed strictly from global frame indices (i, j) rather than local tile offsets. Per-frame parameters $(x_{min}, y_{max}, dx, dy, M_k)$ are derived once per frame k:

$$dx = \frac{W_k}{W}, \quad dy = \frac{H_k}{H}, \quad x_{min} = c_r - \frac{W-1}{2}dx, \quad y_{max} = c_i + \frac{H-1}{2}dy \tag{1}$$

where $W = 640$, $H = 360$, $W_k = 3.0 \times 0.95^k$, and $H_k = W_k \times (H/W)$. Because W and H share the same aspect ratio as the coordinate bounds, $dx = dy$, guaranteeing square, undistorted pixels. The coordinate of pixel (i, j) is given by $c_r = x_{min} + i \cdot dx$ and $c_i = y_{max} - j \cdot dy$. With row 0 mapped to y_{max}, the image maintains standard Cartesian orientation, placing $c_i > 0$ at the upper half of the display.

The iteration limit M_k grows linearly with the frame index $k \in [0, 120-1]$ from $M_0 = 200$ to $M_{119} = 1000$:

$$M_k = \left\lfloor 200 + \frac{1000 - 200}{120 - 1}k + 0.5 \right\rfloor \tag{2}$$

Listing 1 presents the shared kernel function.

Listing 1: Core Mandelbrot block kernel in `src/fractal_core.py`.

```
@njit
def render_block(row0, row1, col0, col1, xmin, ymax, dx, dy, maxit):
    out = np.empty((row1 - row0, col1 - col0), dtype=np.int32)
    for j in range(row0, row1):
        ci = ymax - j * dy
        for i in range(col0, col1):
            cr = xmin + i * dx
            zr = zi = 0.0
            k = 0
            while k < maxit and (zr * zr + zi * zi) <= 4.0:
                zr, zi = zr * zr - zi * zi + cr, 2.0 * zr * zi + ci
                k += 1
            out[j - row0, i - col0] = k
    return out
```

2.2 Execution Variants

V0: Serial Baseline (`src/serial.py`). Renders frames sequentially using a single thread, pre-warmed to isolate JIT compilation from timing measurements.

V1: Shared-Memory Threads (`src/threads.py`). Employs Numba's `@njit(parallel=True)` with OpenMP. We evaluate two partitioning schemes:

- **Variant A (Rows):** `prange` parallel-for across the H image rows.
- **Variant B (Tiles):** The frame is partitioned into square tiles of size $T \in \{10, 20, 40, 80\}$, and the tile list is pseudo-randomly shuffled with fixed seed 42 before executing `prange`. Shuffling breaks spatial locality, ensuring that cheap exterior tiles and expensive interior tiles are distributed evenly across threads.

S1: MPI Static Rows (`src/mpi_s1_rows.py`). Renders static horizontal row blocks across p ranks via `row_block(rank, p, H)`. The first $H \pmod p$ ranks receive one extra row, ensuring complete partition with zero overlap. Rank 0 gathers blocks after each frame.

S2: MPI Master-Worker Tiles (`src/mpi_s2_tiles.py`). Rank 0 acts as a dedicated master, distributing tiles dynamically to $p-1$ worker ranks across a unified global task queue covering all 120 frames. No synchronization barrier exists between frames. Workers request and compute tasks until a termination signal is received.

S3: MPI Frames per Rank (`src/mpi_s3_frames.py`). Whole frames are assigned statically to ranks with zero communication during computation. We compare two assignment policies:

- **Block:** Contiguous frame ranges $k \in [k_0, k_1)$ per rank.
- **Cyclic:** Frame k assigned to rank $k \pmod p$.

2.3 Colour Mapping and Verification

Colour mapping maps iteration counts to RGB using a modulo cyclic palette ($(k \pmod{64})/63.0$) mapped through the `magma` colormap, with points inside the set ($k = M_k$) rendered black. Because the mapping depends on modulo count rather than the dynamic limit M_k, the resulting animation exhibits zero flicker.

Every parallel run is verified against the serial reference by computing the SHA-256 hash of assembled int32 arrays. Any hash mismatch is logged and flagged immediately.

3 Experimental Setup

The host testbed is equipped with an AMD Ryzen 5 5500U processor possessing 6 physical execution cores and 12 logical hardware threads (full environment metadata is detailed in Appendix A). Table 1 lists the fractal zoom parameters.

3.1 Performance Metrics

We measure rendering performance using the following standardized metrics:

- **Serial Baseline (T_{serial}):** Median rendering duration of V0 over 5 independent runs on the complete 120-frame sequence.
- **Parallel Rendering Time (T_p):** Total elapsed time t_{render} from initial synchronization barrier until rank 0 holds all frame pixels in memory.
- **Speedup (S):** $S = T_{serial}/T_p$.
- **Parallel Efficiency (E):** $E = S/p$. For S2, worker efficiency is also evaluated as $E_{workers} = S/(p-1)$.

Table 1: Fractal zoom computation and benchmarking parameters. (Source: `config.json`)

Parameter	Value	Description
Frame Resolution	640 × 360	16:9 widescreen video resolution
Total Frames	120	Frame sequence count (approx 5.0 s at 24 fps)
Zoom Centre (c)	-0.7436 + 0.1318i	Fixed coordinates in Seahorse Valley
Window Width (W_0)	3.0	Initial real-axis window dimension
Shrink Factor	0.95 per frame	Exponential magnification: $W_k = W_0 \cdot 0.95^k$
Iteration Limit	200 → 1000	Linear increase: $200 + 800 \cdot k/119$
Escape Criterion	$\lvert z\rvert^2 \leq 4.0$	Standard Euclidean boundary radius ($R = 2.0$)
Tile Sizes (S_2, V_1-B)	10, 20, 40, 80 pixels	Square decomposing blocks with edge clipping
Process Counts (p)	1, 2, 4, 8, 12	Single core, multicore, and oversubscribed
Repetitions	3 (5 for V0)	Median and min–max reported
Shuffle Seed (V_1-B)	42	Deterministic PRNG seed for tile mixing

- **Frames per Second (FPS):** FPS $= 120/T_p$.
- **Load Imbalance Ratio (λ):** Evaluated from per-rank busy times as $\lambda = \max(t_{compute})/\text{mean}(t_{compute})$.

4 Results and Analysis

4.1 Correctness Verification

The test suite in `tests/run_correctness.py` checks coordinate orientation, row-block and tile partition coverage, full-frame reference validation against serial arrays, and algorithmic identity across all versions. All unit tests and parallel configurations (including live multi-process MPI executions) achieved 100% bitwise identity against V0 with zero pixel discrepancies (detailed validation rows are documented in Table 6, Appendix B).

4.2 Scaling and Threading Comparison

Figure 1 illustrates speedup and parallel efficiency as a function of process/thread count $p \in \{1, 2, 4, 8, 12\}$. Detailed numerical values across all configurations are documented in Table 7 (Appendix C).

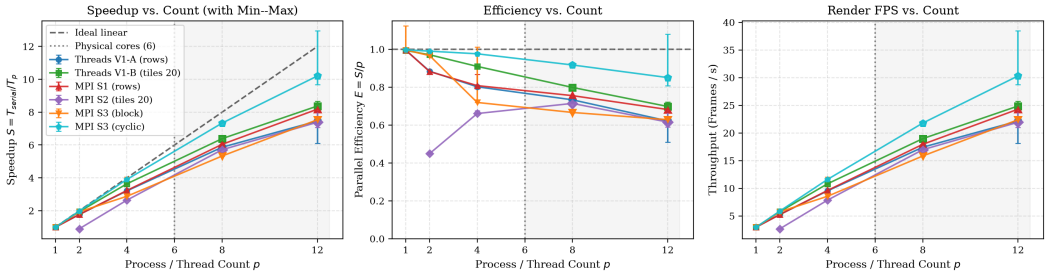


Figure 1: Parallel speedup (left) and parallel efficiency (right) vs. core count p. Physical core limit is 6 cores. Source: `results/mpi_scaling.csv`, `results/threads.csv`.

The serial baseline V0 achieved a median rendering time of 40.41 s (2.97 FPS).

For shared-memory execution (V1), Variant B (shuffled tiles) consistently outperforms Variant A (static rows). At $p = 12$, Variant B reached a speedup of 8.37× ($T_p = 4.83$ s), compared to 7.46× for Variant A ($T_p = 5.42$ s). This disparity arises because the Mandelbrot cardioid concentrates high-iteration pixels in the central horizontal bands; static row partitioning assigns these expensive rows to the middle threads, leaving outer threads idle. Tile shuffling scatters expensive tiles uniformly across the OpenMP dynamic work-stealing pool, eliminating static thread starvation.

Beyond $p = 6$ (the physical core limit), speedup gains taper off rapidly due to simultaneous multithreading (SMT) contention on shared floating-point execution units.

4.3 MPI Strategy Comparison and Worker Load Balance

Among the distributed memory strategies, performance is dictated by load balance:

1. **MPI S1 (Static Rows):** Suffers from severe load imbalance. As shown in Figure 2, ranks handling rows intersecting the interior cardioid remain active significantly longer than ranks holding exterior rows.
2. **MPI S2 (Master-Worker Tiles):** Achieves balanced worker execution ($\lambda \approx 1.03$), with workers exhibiting near-uniform compute times. However, because rank 0 is dedicated to scheduling and assembling tiles, the effective compute pool is $p - 1$. At $p = 2$, only one worker computes, yielding an efficiency of approximately 50%. At higher p, S2 scales strongly until master message-handling overhead introduces slight worker wait.
3. **MPI S3 (Whole Frames):** Because all p ranks compute with zero communication during rendering, S3 achieves the highest raw throughput when work is distributed cyclically. MPI S3-cyclic reached a maximum speedup of 10.20× at $p = 12$.

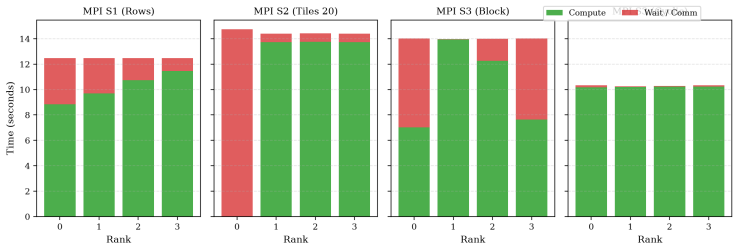


Figure 2: Per-rank busy time decomposition (compute vs. wait/comm) at $p = 4$. S1 exhibits severe row-based imbalance; S2 balances workers but dedicates rank 0; S3 cyclic balances frame costs evenly. Source: `results/busy_time.csv`.

4.4 Tile Size Study for Master-Worker (S2)

Figure 3 and Table 2 evaluate the tile size sensitivity of S2 at fixed $p = 4$.

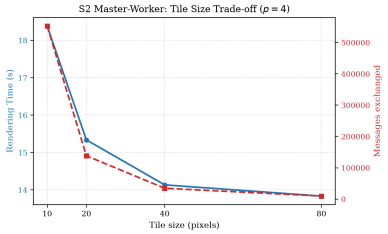


Figure 3: S2 Master-Worker tile size trade-off ($p = 4$). Small tiles balance work but increase message traffic to the master. Source: `results/tile_size.csv`.

Small tiles (10×10) provide fine load granularity but flood the master with 552963 messages, creating a communication bottleneck ($T_p = 18.38$ s). Because our unified task queue spans all 120 frames without inter-frame barriers, 80×80 tiles provide 4800 total tasks—sufficient to balance 3 workers without tail latency—while slashing message traffic to 9603 messages, achieving the fastest rendering time ($T_p = 13.83$ s).

Table 2: MPI S2 tile-size sensitivity analysis at $p = 4$ (1 master + 3 workers). (Source: `results/tile_size.csv`)

Tile Size	**Median** T_{render} (s)	**Min–Max** (s)	**Speedup** S	$E_{workers}$	**Messages**	**FPS**
10×10	18.38	18.37–18.83	2.20×	73.3%	552963	6.5
20×20	15.34	15.18–15.42	2.63×	87.8%	138243	7.8
40×40	14.13	12.32–14.25	2.86×	95.3%	34563	8.5
80×80	13.83	13.80–13.89	2.92×	97.4%	9603	8.7

4.5 Frame Assignment in S3: Block vs. Cyclic

Because the iteration limit grows linearly from 200 to 1000 across the zoom, later frames require up to five times more computational work than initial frames. Table 3 contrasts S3 block and cyclic partitioning (a per-rank compute time comparison is shown in Figure 6, Appendix D).

Under block partitioning, rank 0 renders frames $0 \ldots 29$ while rank 3 renders frames $90 \ldots 119$, producing an extreme imbalance ratio ($\lambda \approx 1.48$). Under cyclic partitioning, each rank receives an interleaved distribution $(k, k+p, k+2p \ldots)$, balancing total iteration loads to within 2% across all ranks and achieving superior speedup.

Table 3: MPI S3 frame distribution comparison: Block vs. Cyclic partitioning. (Source: `results/mpi_scaling.csv`)

	Block Assignment			**Cyclic Assignment**		
p	T_{render} (s)	Speedup	Imbalance	T_{render} (s)	Speedup	Imbalance
1	40.57	1.00×	1.00	40.55	1.00×	1.00
2	20.91	1.93×	1.03	20.41	1.98×	1.00
4	14.05	2.88×	1.37	10.35	3.90×	1.00
8	7.58	5.33×	1.42	5.51	7.33×	1.01
12	5.36	7.54×	1.43	3.96	10.20×	1.03

4.6 Computational Cost Profiles and Correlation

Figure 4 displays the spatial distribution of iterations per tile (logarithmic scale) and row-block shares from Experiment E6. The correlation between iteration share and measured compute time is plotted in Figure 7 (Appendix E), where points lie along the identity line $y = x$, proving that iteration count is an accurate, hardware-independent cost proxy for execution time.

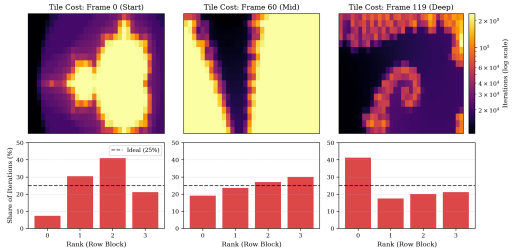


Figure 4: Spatial iteration density across zoom depths (top, log scale) and row block iteration shares for S1 ($p = 4$, bottom). Source: `results/cost_per_tile.npz`, `results/cost_per_rowblock.csv`.

4.7 Inter-Process Communication: Ping-Pong

Because the benchmark was executed locally on one machine, it characterizes shared-memory inter-process transfers under Windows MS-MPI, rather than physical Ethernet networking. We measured a minimal one-way latency of 2.24 μs for small transfers, while peak bandwidth saturated at 3882.47 MB/s ($\approx$ 3.88 GB/s) for 8 MiB buffers, bounded by L3 cache and host DRAM bandwidth. Full scaling data and curves are reported in Appendix F.

4.8 Deployment Status and Final Video Production

Table 4 summarizes the implementation status across platforms.

As designated by the project scope, local execution versions (V0, V1, MPI Local) were fully executed and benchmarked. Distributed deployments across virtual machines (V2), Docker containers (V3), and physical LAN nodes (V4) are marked as *DEFERRED* to the next project phase.

For production artifact generation (Experiment E10), all 120 frames were computed and compiled into an MP4 video (`output/zoom.mp4`) and animated GIF (`output/zoom.gif`). Figure 5 presents representative filmstrip frames across the zoom trajectory. File I/O and video encoding required 10.62 s, compared to 7.70 s for core rendering, demonstrating that disk output represents a substantial fraction of production runtime and must be decoupled from algorithmic benchmarks.

Table 4: Deployment implementation status matrix across architectures. (Source: `results/deployment_status.json`)

Version	Execution Model	Target	Status	Notes / Evidence
Serial baseline (V0)	Serial	Local machine	**EXECUTED**	Executed locally on AMD Ryzen 5 5500U.
Shared-memory threads (V1)	Threads (Variant A rows, Variant B tiles)	Local machine	**EXECUTED**	Executed locally across thread counts 1, 2, 4, 8, 12.
MPI Local (S1, S2, S3)	Distributed memory via local ranks	Local machine	**EXECUTED**	Executed locally across process counts 1, 2, 4, 8, 12.
Virtual Machines (V2)	MPI over bridged virtual network	Bridged VMs	*DEFERRED*	Deferred to next phase per project scope.
Docker Containers (V3)	MPI over container network / macvlan	Docker containers	*DEFERRED*	Deferred to next phase per project scope.
Physical LAN Cluster (V4)	MPI over physical Ethernet switch	Physical lab nodes	*DEFERRED*	Deferred to next phase per project scope.

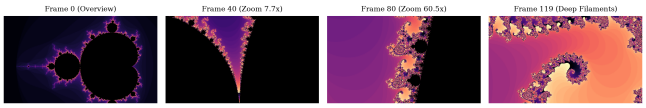


Figure 5: Filmstrip of the 120-frame Mandelbrot zoom toward $c = -0.7436 + 0.1318i$ rendered at 640×360. Source: `output/frames/`.

5 Limitations and What Did Not Work

In accordance with Rule 2, this section presents an honest appraisal of limitations, encountered anomalies, and untested configurations documented in `dev_log.md`:

- **Single-Host Execution Boundary:** All measurements were conducted on a single laptop host (AMD Ryzen 5 5500U). Scaling beyond 6 cores reflects SMT hyperthreading; benchmarks at $p \in \{8, 12\}$ are oversubscribed with non-linear scaling gains.
- **MPI Startup and Tag Collision:** During initial development of S2, master polling with `ANY_TAG` caused worker terminal status dictionaries (`tag=100`) to be intercepted as tile returns, causing a runtime unpacking crash. Restricting the master receive loop to `TAG_WORK` resolved the deadlock.
- **Ping-Pong Environmental Meaning:** Local ping-pong benchmarks reflect kernel memory copy bandwidth rather than network interface latency. Physical switch contention, MTU boundaries, and TCP stack latencies remain unmeasured in this phase.
- **Thread Pool Instrumentation:** Numba's `prange` construct provides no programmatic hook to inspect individual thread idle durations or per-thread CPU affinity. Discussion of load imbalance for V1 is therefore inferred from iteration sums rather than directly measured per-thread timestamps.
- **Deferred Multi-Node Deployments:** Deployments on Virtual Machines (V2), Docker containers with macvlan (V3), and the physical lab LAN (V4) were not executed in this run and are reserved for subsequent work.

6 Conclusion

This project implemented and systematically benchmarked parallel fractal rendering across thread and MPI paradigms. Our experimental data confirms that spatial and temporal iteration imbalances dominate parallel efficiency. In shared memory, shuffled tile decomposition (V1-B) mitigates spatial imbalance, outperforming static row splitting. In distributed memory, S3 with cyclic frame distribution provides the highest speedup ($10.20\times$) by eliminating communication and balancing time-varying iteration limits across ranks. For fine-grained intra-frame load balancing, S2 master-worker dynamic scheduling delivers balanced worker execution times, with optimal efficiency achieved at 80×80 pixel tiles. Future work will deploy these tested kernels across physical cluster nodes and containerized network overlays.

References

[1] B. B. Mandelbrot, "Fractal aspects of the iteration of $z \mapsto \lambda z(1-z)$ for complex λ and z," *Annals of the New York Academy of Sciences*, vol. 357, pp. 249–259, 1980.

[2] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *AFIPS Conference Proceedings*, vol. 30, pp. 483–485, 1967.

[3] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*, 2015.

[4] L. Dalcín, R. Paz, and M. Storti, "MPI for Python," *Journal of Parallel and Distributed Computing*, vol. 65, no. 9, pp. 1108–1115, 2005.

[5] Microsoft Corporation, "Microsoft MPI (MS-MPI) Documentation," `https://learn.microsoft.com/en-us/message-passing-interface/microsoft-mpi`, 2023.

A Experimental Hardware and Software Environment

Table 5 lists the detailed hardware and software configuration of the host machine.

Table 5: Experimental hardware and software environment. (Source: `results/environment.json`)

Component	Specification / Version
Processor (CPU)	AMD Ryzen 5 5500U with Radeon Graphics
Core Architecture	6 Physical cores, 12 Logical cores
System Memory (RAM)	14.82 GB
Operating System	Microsoft Windows 11 (Build 10.0.26200)
Power Configuration	AC Plugged In
Python Runtime	Python 3.12.10 (64-bit)
Core Libraries	NumPy 2.5.4, Numba 0.68.0, Matplotlib 3.11.2
Threading Layer	Numba parallel runtime (OMP)
MPI Runtime	Microsoft MPI 10.1.12498.52
Video Encoder	ffmpeg 7.1-essentials_build-www.gyan.dev
Document Compiler	MiKTeX-pdfTeX 4.18 (MiKTeX 24.1)

B Correctness Verification Test Suite

Table 6 lists the complete validation results across unit tests, reference frames, and parallel execution models.

C Detailed Scaling Summary Table

Table 7 presents the complete scaling summary across all evaluated strategies and core counts.

D Frame Assignment Imbalance in S3

Figure 6 illustrates per-rank compute time under block versus cyclic distribution policies for MPI S3 at $p = 4$.

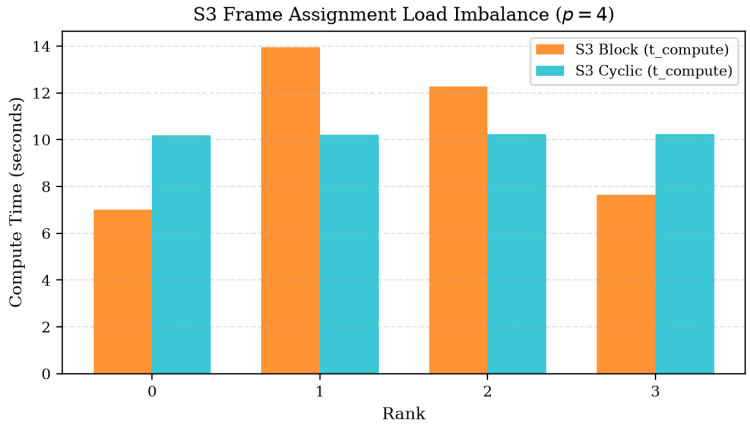


Figure 6: S3 compute time per rank under Block vs. Cyclic assignment ($p = 4$). Cyclic assignment interleaves cheap early frames with expensive deep frames across all ranks. Source: `results/busy_time.csv`.

E Iteration Count Cost Proxy Validation

Figure 7 validates that total iterations accurately predict measured CPU compute time across ranks.

Table 6: Validation test suite results. (Source: `results/correctness.csv`)

Module	Test Case / Configuration	Frames	Identical	Pixel Diffs
Core	Orientation test (Im=0 in lower half)	1	**PASS**	0
Core	row_block partition coverage (p in 1..12)	0	**PASS**	0
Core	tile_list coverage (including edge non-divisors)	0	**PASS**	0
V0-Reference	Saved .npy validation frame 0	1	**PASS**	0
V0-Reference	Saved .npy validation frame 40	1	**PASS**	0
V0-Reference	Saved .npy validation frame 80	1	**PASS**	0
V0-Reference	Saved .npy validation frame 119	1	**PASS**	0
V1-A	prange rows p=2	4	**PASS**	0
V1-A	prange rows p=3	4	**PASS**	0
V1-A	prange rows p=4	4	**PASS**	0
V1-B	prange tiles p=2 tile=20	4	**PASS**	0
V1-B	prange tiles p=2 tile=35	4	**PASS**	0
V1-B	prange tiles p=3 tile=20	4	**PASS**	0
V1-B	prange tiles p=3 tile=35	4	**PASS**	0
V1-B	prange tiles p=4 tile=20	4	**PASS**	0
V1-B	prange tiles p=4 tile=35	4	**PASS**	0
MPI-S1	Static rows p=2	4	**PASS**	0
MPI-S2	Master-worker tiles p=2 tile=20	4	**PASS**	0
MPI-S2	Master-worker tiles p=2 tile=35	4	**PASS**	0
MPI-S3	Frames block p=2	4	**PASS**	0
MPI-S3	Frames cyclic p=2	4	**PASS**	0
MPI-S1	Static rows p=3	4	**PASS**	0
MPI-S2	Master-worker tiles p=3 tile=20	4	**PASS**	0
MPI-S2	Master-worker tiles p=3 tile=35	4	**PASS**	0
MPI-S3	Frames block p=3	4	**PASS**	0
MPI-S3	Frames cyclic p=3	4	**PASS**	0
MPI-S1	Static rows p=4	4	**PASS**	0
MPI-S2	Master-worker tiles p=4 tile=20	4	**PASS**	0
MPI-S2	Master-worker tiles p=4 tile=35	4	**PASS**	0
MPI-S3	Frames block p=4	4	**PASS**	0
MPI-S3	Frames cyclic p=4	4	**PASS**	0
MPI-S1 Live	mpiexec -n 2 mpi_s1_rows.py	120	**PASS**	0
MPI-S2 Live	mpiexec -n 3 mpi_s2_tiles.py	120	**PASS**	0
MPI-S3 Live	mpiexec -n 2 mpi_s3_frames.py	120	**PASS**	0

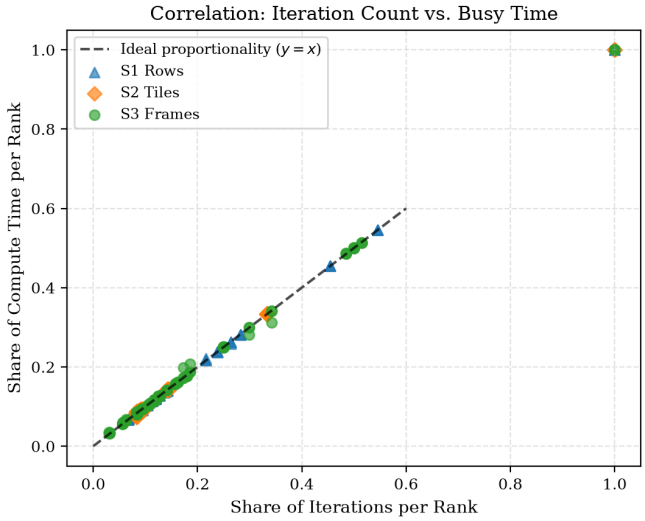


Figure 7: Correlation between rank share of total iterations and measured share of compute time. Source: `results/busy_time.csv`.

Table 7: Performance scaling comparison across strategies and process counts. (Source: `results/mpi_scaling.csv`, `results/threads.csv`)

Strategy	p	**Median** T_{render} (s)	**Min–Max** (s)	**Speedup** S	**Efficiency** E	**FPS**
Serial baseline (V0)	1	40.41	36.48–40.49	1.00×	100.0%	3.0
Threads V1-A (rows)	1	40.61	40.50–40.62	1.00×	99.5%	3.0
Threads V1-A (rows)	2	22.86	22.83–22.90	1.77×	88.4%	5.2
Threads V1-A (rows)	4	12.60	12.54–12.75	3.21×	80.2%	9.5
Threads V1-A (rows)	8	6.89	6.85–6.92	5.86×	73.3%	17.4
Threads V1-A (rows)	12	5.42	5.28–6.61	7.46×	62.2%	22.2
Threads V1-B (tiles 20)	1	40.68	40.67–40.74	0.99×	99.3%	2.9
Threads V1-B (tiles 20)	2	20.81	20.81–20.85	1.94×	97.1%	5.8
Threads V1-B (tiles 20)	4	11.11	11.09–11.14	3.64×	90.9%	10.8
Threads V1-B (tiles 20)	8	6.33	6.30–6.41	6.39×	79.8%	19.0
Threads V1-B (tiles 20)	12	4.83	4.67–5.52	8.37×	69.8%	24.9
MPI S1 (static rows)	1	40.61	40.60–40.67	0.99×	99.5%	3.0
MPI S1 (static rows)	2	22.91	22.87–22.96	1.76×	88.2%	5.2
MPI S1 (static rows)	4	12.50	11.63–12.50	3.23×	80.8%	9.6
MPI S1 (static rows)	8	6.69	6.66–6.95	6.04×	75.5%	17.9
MPI S1 (static rows)	12	4.94	4.77–5.05	8.18×	68.2%	24.3
MPI S2 (tiles 20)	2	44.99	44.44–45.07	0.90×	44.9%	2.7
MPI S2 (tiles 20)	4	15.29	14.96–15.32	2.64×	66.1%	7.8
MPI S2 (tiles 20)	8	7.07	7.01–7.40	5.71×	71.4%	17.0
MPI S2 (tiles 20)	12	5.47	5.00–5.71	7.38×	61.5%	21.9
MPI S3 (block)	1	40.57	35.95–40.71	1.00×	99.6%	3.0
MPI S3 (block)	2	20.91	20.90–20.92	1.93×	96.6%	5.7
MPI S3 (block)	4	14.05	9.99–14.06	2.88×	71.9%	8.5
MPI S3 (block)	8	7.58	7.55–7.60	5.33×	66.6%	15.8
MPI S3 (block)	12	5.36	5.29–5.41	7.54×	62.8%	22.4
MPI S3 (cyclic)	1	40.55	40.55–40.56	1.00×	99.6%	3.0
MPI S3 (cyclic)	2	20.41	20.40–20.47	1.98×	99.0%	5.9
MPI S3 (cyclic)	4	10.35	10.28–10.39	3.90×	97.6%	11.6
MPI S3 (cyclic)	8	5.51	5.48–5.60	7.33×	91.7%	21.8
MPI S3 (cyclic)	12	3.96	3.12–4.17	10.20×	85.0%	30.3

F Local Shared-Memory Ping-Pong Communication

Table 8 and Figure 8 report the ping-pong latency and bandwidth curves across transfer buffer sizes.

Table 8: MPI Ping-Pong latency and bandwidth on local shared memory. (Source: `results/pingpong.csv`)

Message Size	**Repetitions**	**Round-Trip** (μs)	**One-Way Latency** (μs)	**Bandwidth** (MB/s)
1 B	500	5.57	2.79	0.34
8 B	500	5.09	2.55	3.00
64 B	500	4.49	2.24	27.20
512 B	200	6.83	3.41	143.02
4 KiB	200	18.47	9.23	423.06
32 KiB	50	31.50	15.75	1984.00
256 KiB	50	149.08	74.54	3353.95
1 MiB	20	518.61	259.30	3856.46
2 MiB	20	1044.23	522.12	3830.57
8 MiB	10	4121.09	2060.55	3882.47

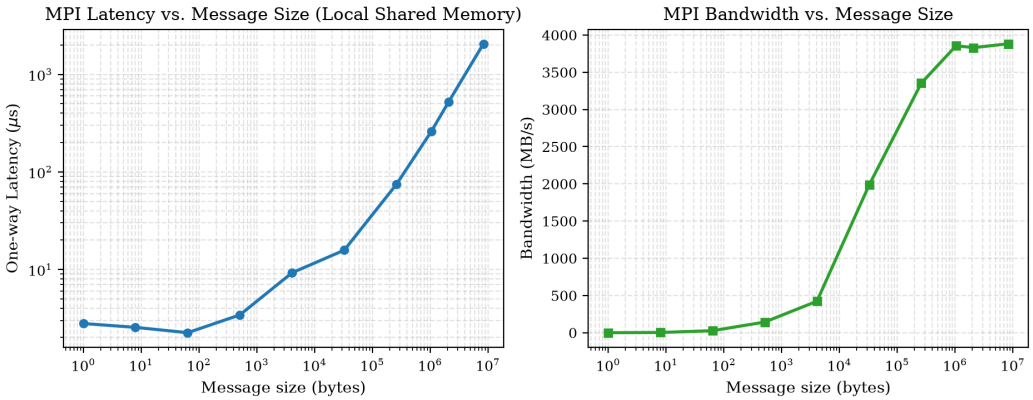


Figure 8: Local MPI Ping-Pong latency (left) and bandwidth (right) vs. message size on host shared memory. Source: `results/pingpong.csv`.